Docker Compose



Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a concept and practice in software engineering and DevOps that involves managing and provisioning infrastructure using code and automation rather than manual processes. It aims to treat infrastructure as if it were software, enabling developers and operations teams to create, modify, and manage infrastructure resources through code, just as they would with application code.

Traditionally, setting up and configuring infrastructure (servers, networks, databases, etc.) has been a manual and error-prone process. With Infrastructure as Code, this process becomes more efficient, reliable, and scalable.

IaC

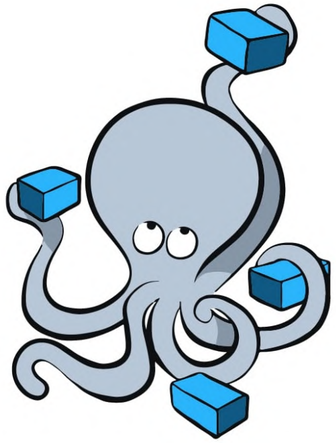
Key benefits of adopting IaC include:

1. **Version control**: Infrastructure code can be stored in version control systems (e.g., Git), enabling tracking of changes, collaboration, and rollbacks.
2. **Reproducibility**: IaC allows for the precise recreation of environments across different stages of development, testing, and production.
3. **Scalability**: Infrastructure provisioning can be automated, making it easier to scale up or down as needed.
4. **Consistency**: IaC ensures that the same configuration is applied consistently across all environments, reducing the risk of misconfigurations and inconsistencies.
5. **Faster deployment**: Automated provisioning reduces manual intervention and speeds up the deployment process.
6. **Cost optimization**: IaC allows teams to define and manage infrastructure resources optimally, leading to cost savings.

IaC


Templates
Scripts
Policies
ANSIBLE
Terraform
CHEF
puppet
Network
Application
Storage
Security
Cloud
Infrastructure

Docker Compose



docker
Compose

Docker Compose is a tool provided by Docker that allows you to define and manage multi-container Docker applications. It provides a way to define all the services, networks, and volumes required for your application in a single YAML file called **docker-compose.yml**. This file allows you to specify the configuration and relationships between various containers, making it easier to deploy and manage complex applications.

Docker Compose can run applications on a **single host only**.

Docker Compose

Using Docker Compose, you can define the following aspects of your application:

- **Services**: These are the individual containers that make up your application. Each service can be based on a specific Docker image and can have its own configuration options.
- **Networks**: Compose allows you to define custom networks that enable communication between containers. By default, containers within the same Compose file are connected to the default network and can communicate with each other using service names.
- **Volumes**: You can specify data volumes that persist data between container restarts. This is useful for databases and other stateful applications.

Nginx



Nginx (pronounced "engine x") is a popular open-source web server, reverse proxy server, and load balancer. It's well-known for its high performance, stability, and low resource usage, making it a top choice for serving static content, handling web application traffic, and improving server efficiency. Nginx is widely used by many large-scale websites, web applications, and various projects worldwide.

Nginx

Key features of Nginx include:

- **Web Server**: Nginx can serve static content like HTML, CSS, JavaScript files, images, and videos directly to clients, improving the delivery speed and reducing the load on application servers.
- **Reverse Proxy**: Nginx can act as a reverse proxy, forwarding client requests to backend servers and passing responses back to clients. This allows for load balancing, caching, SSL/TLS termination, and handling various protocols.
- **Load Balancing**: Nginx can distribute incoming client requests across multiple backend servers to achieve better performance, scalability, and fault tolerance for web applications.
- **SSL/TLS Termination**: Nginx can handle SSL/TLS encryption and decryption, offloading the resource-intensive SSL operations from the application servers.
- **Caching**: Nginx can cache static content and even dynamic content in some cases, reducing the load on backend servers and improving response times for clients.
- **Security**: Nginx includes various security features, such as access control, request and rate limiting, and support for SSL/TLS protocols to ensure secure communication.
- **HTTP/2 and HTTP/3 Support**: Nginx supports the latest HTTP protocols, providing improved performance and efficiency over older versions.
- **URL Rewriting**: Nginx can rewrite URLs, allowing for cleaner, more user-friendly URLs or redirecting traffic based on specific patterns.

Reverse Proxy

A reverse proxy is a server or software component that sits between client devices (such as web browsers) and backend servers. Its primary function is to handle incoming client requests and forward them to the appropriate backend server, acting on behalf of the server. The responses from the backend server are then sent back to the clients through the reverse proxy, effectively acting as an intermediary or gateway.

The term "reverse" in "reverse proxy" refers to the fact that it operates on the server-side, as opposed to a regular proxy that typically works on the client-side.

Reverse Proxy

Here's how a reverse proxy functions:

1. **Client Request**: When a client device (e.g., a web browser) makes a request to access a particular resource (like a web page or file), the request is sent to the reverse proxy.
2. **Routing**: The reverse proxy examines the request and determines which backend server should handle it. This decision can be based on various factors, such as load balancing algorithms, server health checks, or URL patterns.
3. **Forwarding**: After identifying the appropriate backend server, the reverse proxy forwards the client's request to that server on behalf of the client.
4. **Backend Server Response**: The backend server processes the request and generates a response.
5. **Response to Client**: The reverse proxy receives the response from the backend server and forwards it back to the client device that initiated the request.

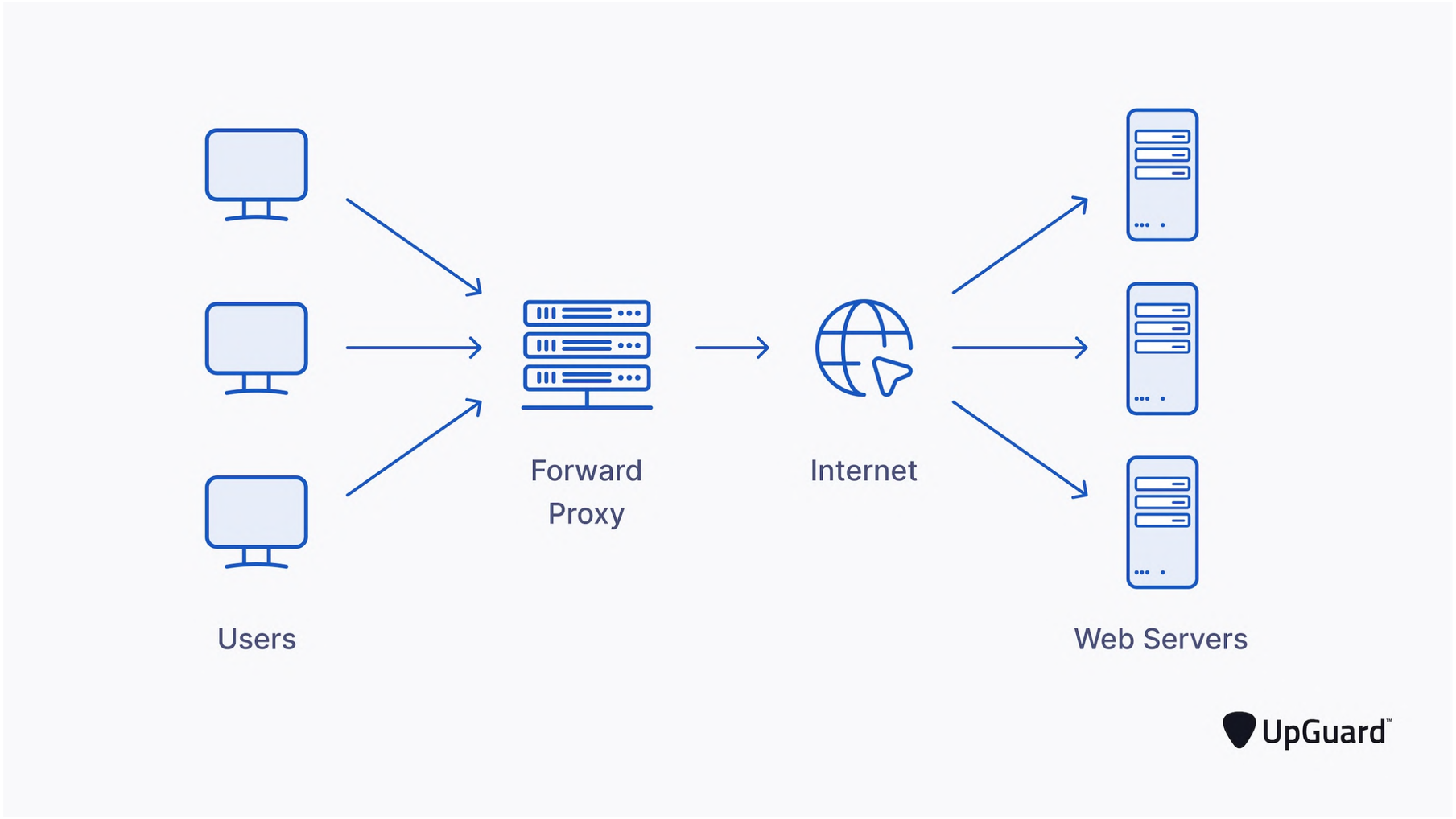
Reverse Proxy

Reverse proxies offer several benefits, including:

1. **Load Distribution**: They can distribute client requests among multiple backend servers, helping to balance the load and prevent individual servers from becoming overwhelmed.
2. **Security**: Reverse proxies can act as a buffer between clients and backend servers, adding an extra layer of security by hiding the internal server structure and potentially filtering and blocking malicious requests.
3. **Caching**: Reverse proxies can cache responses from backend servers, improving response times for frequently requested resources and reducing the load on backend servers.
4. **SSL Termination**: They can handle SSL/TLS encryption and decryption, offloading this computationally intensive task from backend servers.
5. **Content Compression**: Reverse proxies can compress content before sending it to clients, reducing bandwidth usage and improving page load times.

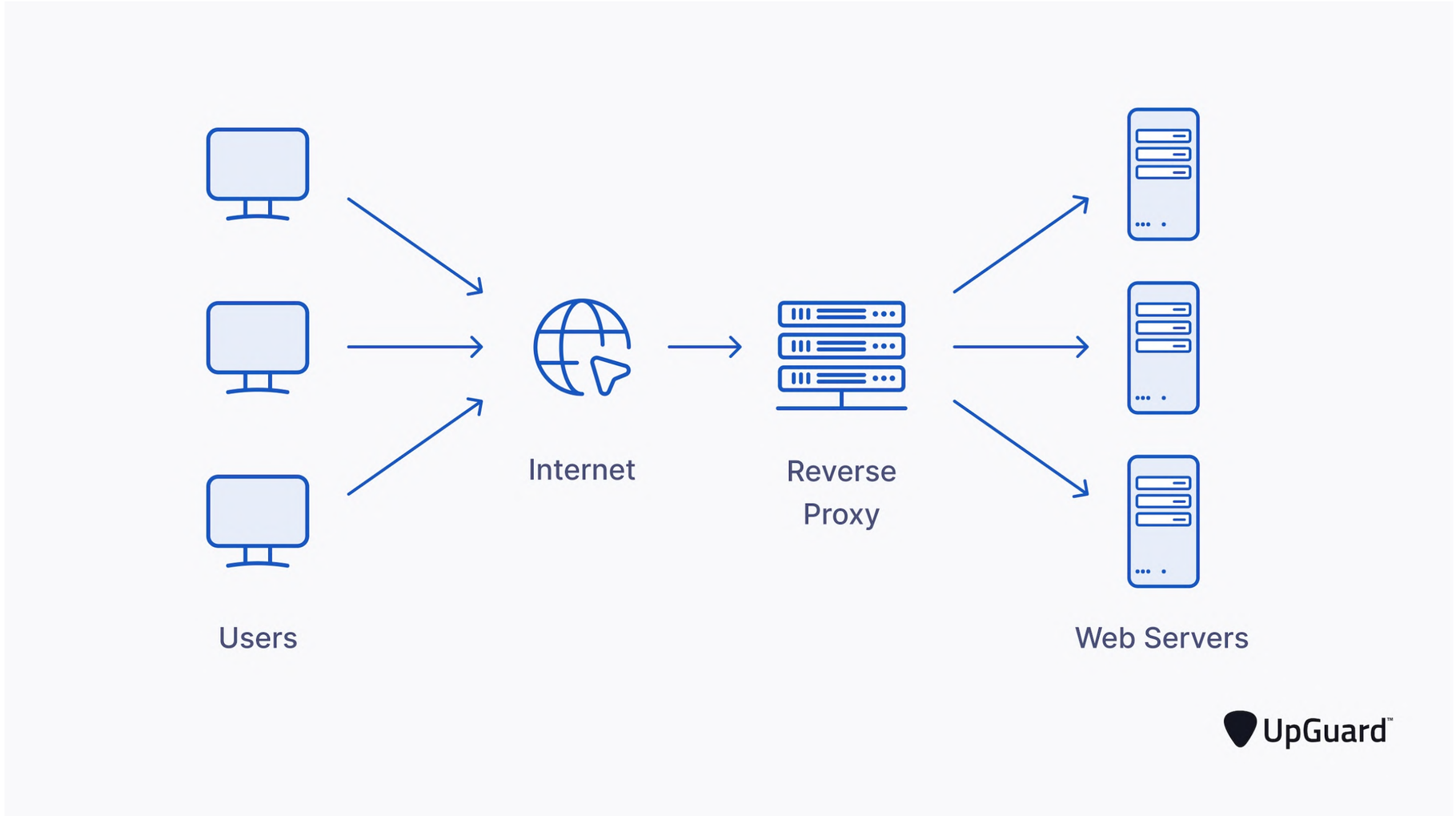
DRIM

Forward Proxy



DRIM

Reverse Proxy

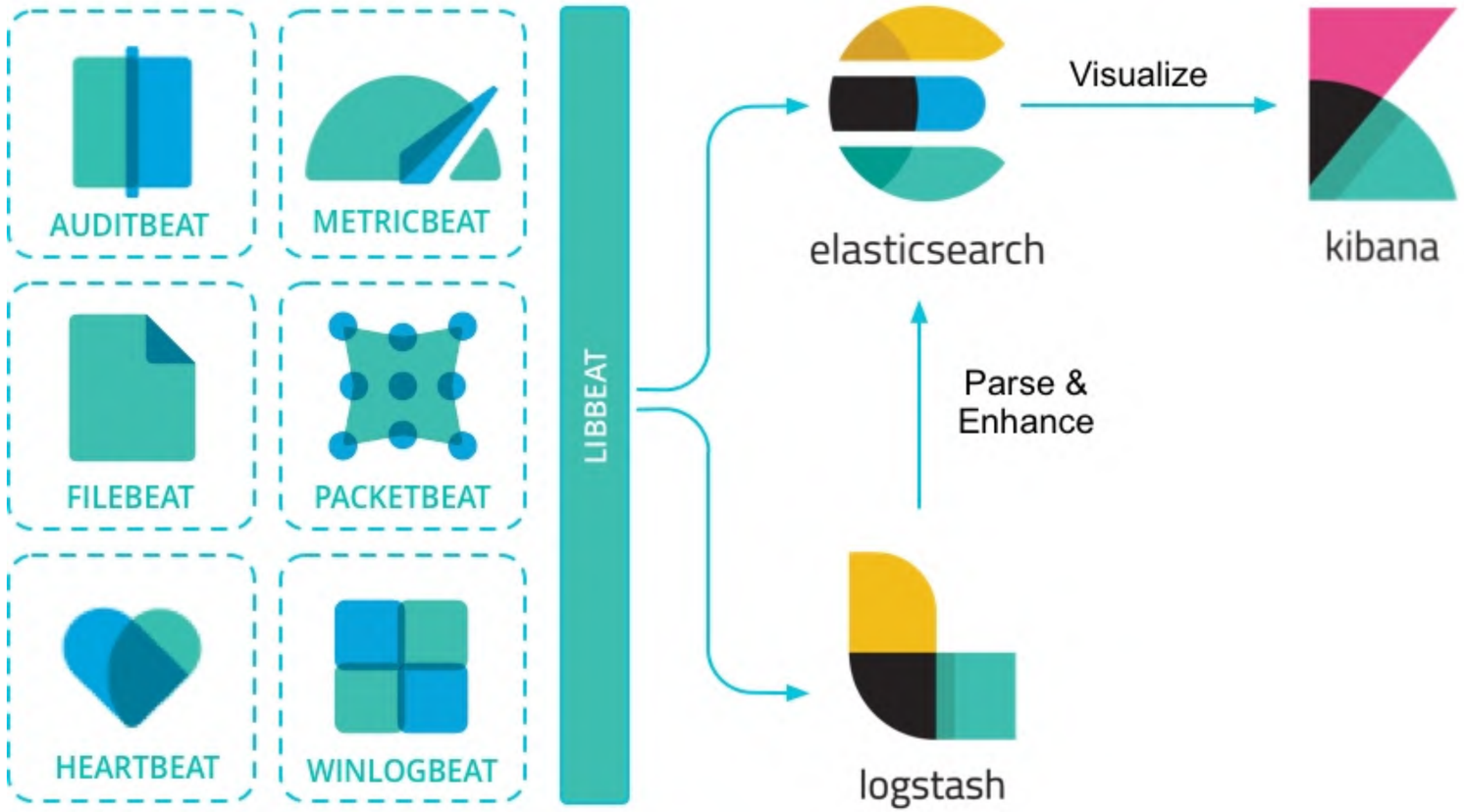


ELK

ELK refers to a popular open-source software stack that is widely used for log management and analysis. ELK is an acronym that stands for:

- **E**lasticsearch: It is a distributed, scalable, and highly available search and analytics engine. Elasticsearch is designed to store, index, and search large volumes of data in real-time. It is the heart of the ELK stack, providing fast and flexible search capabilities for log data.
- **L**ogstash: Logstash is an open-source data processing pipeline that ingests, processes, and transforms log data from various sources before sending it to Elasticsearch for storage and analysis. It allows you to collect logs from different systems, parse them into a structured format, and enrich them if necessary.
- **K**ibana: Kibana is a web-based data visualization and exploration tool that works in conjunction with Elasticsearch. It provides a user-friendly interface to create interactive dashboards, charts, and graphs based on the data stored in Elasticsearch. With Kibana, users can explore log data, monitor system performance, and gain insights from the collected logs.

ELK


AUDITBEAT
METRICBEAT
FILEBEAT
PACKETBEAT
HEARTBEAT
WINLOGBEAT
LIBBEAT
elasticsearch
Visualize
kibana
Parse &
Enhance
logstash

DRIM

Thank you! Questions?

